Instruction-Level Parallelism (ILP)

Instructor: Jeongseob Ahn (안정섭)

E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

Instruction-Level Parallelism (ILP)

- Pipelining: executing multiple instructions in parallel
- To increase ILP
  - Deeper pipeline
    - Less work per stage ⇒ shorter clock cycle
  - Multiple issue
    - Replicate pipeline stages ⇒ multiple pipelines
    - Start multiple instructions per clock cycle
    - CPI < 1, so use Instructions Per Cycle (IPC)
    - e.g., 4GHz 4-way multiple-issue
      - 16 BIPS, peak CPI = 0.25, peak IPC = 4
    - But dependencies reduce this in practice

Multiple Issue

- Static multiple issue
  - Compiler groups instructions to be issued together
  - Packages them into “issue slots”
  - Compiler detects and avoids hazards
- Dynamic multiple issue
  - CPU examines instruction stream and chooses instructions to issue each cycle
  - Compiler can help by reordering instructions
  - CPU resolves hazards using advanced techniques at runtime

Speculation

- `Guess` what to do with an instruction
  - Start operation as soon as possible
  - Check whether guess was right
    - If so, complete the operation
    - If not, roll-back and do the right thing
- Common to static and dynamic multiple issue
- Examples
  - Speculate on branch outcome
    - Roll back if path taken is different
  - Speculate on load
    - Roll back if location is updated

Compiler/Hardware Speculation

- Compiler can reorder instructions
  - e.g., move load before branch
  - Can include “fix-up” instructions to recover from incorrect guess
- Hardware can look ahead for instructions to execute
  - Buffer results until it determines they are actually needed
  - Flush buffers on incorrect speculation

Speculation and Exceptions

- What if exception occurs on a speculatively executed instruction?
  - e.g., speculative load before null-pointer check
- Static speculation
  - Can add ISA support for deferring exceptions
- Dynamic speculation
  - Can buffer exceptions until instruction completion (which may not occur)

Static Multiple Issue

- Compiler groups instructions into “issue packets”
  - Group of instructions that can be issued on a single cycle
  - Determined by pipeline resources required
- Think of an issue packet as a very long instruction
  - Specifies multiple concurrent operations
  - $\Rightarrow$ Very Long Instruction Word (VLIW)

Scheduling Static Multiple Issue

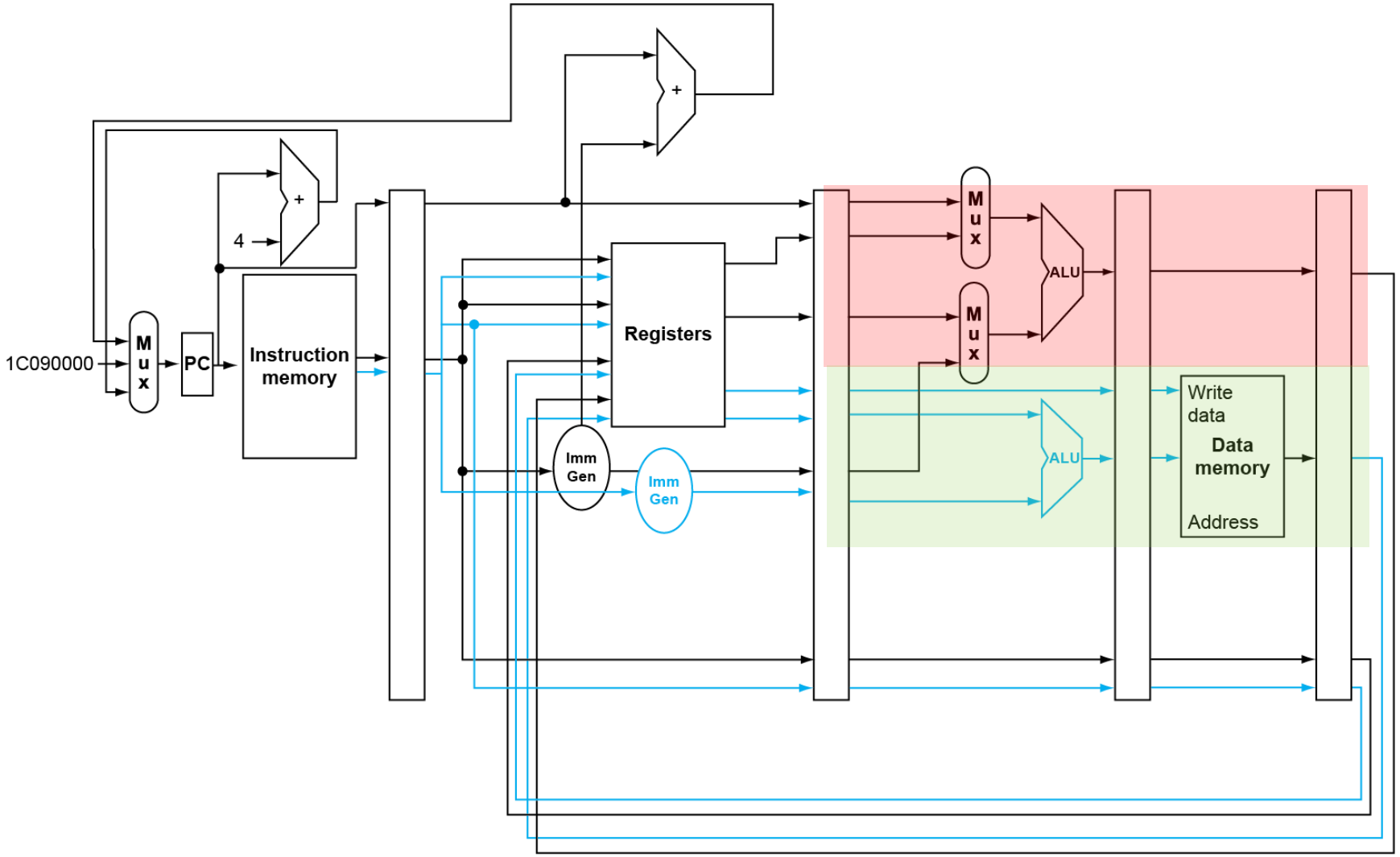
- Compiler must remove some/all hazards
  - Reorder instructions into issue packets
  - No dependencies with a packet
  - Possibly some dependencies between packets
    - Varies between ISAs; compiler must know!
  - Pad with **nop** if necessary

RISC-V with Static Dual Issue

- Two-issue packets
  - One ALU/branch instruction
  - One load/store instruction
  - 64-bit aligned
    - ALU/branch, then load/store
    - Pad an unused instruction with `nop`

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

RISC-V with Static Dual Issue



Hazards in the Dual-Issue RISC-V

- More instructions executing in parallel
- EX data hazard
  - Forwarding avoided stalls with single-issue
  - Now can't use ALU result in load/store in same packet

```
add  x10, x0, x1
load x2, 0(x10)
```

    - Split into two packets, effectively a stall
- Load-use hazard
  - Still one cycle use latency, which means the next two instructions cannot use the load result
- More aggressive scheduling required

Scheduling Example

- Schedule this for dual-issue RISC-V

```
Loop: ld   x31, 0(x20)      # x31=array element
      add  x31, x31, x21    # add scalar in x21
      sd   x31, 0(x20)      # store result
      addi x20, x20,-8      # decrement pointer
      blt  x22, x20, Loop   # branch if x22 < x20
```

	ALU/branch	Load/store	cycle
Loop:	nop	ld x31, 0(x20)	1
	addi x20, x20, -8	nop	2
	add x31, x31, x21	nop	3
	blt x22, x20, Loop	sd x31, 8(x20)	4

  - IPC = 5/4 = 1.25 (c.f. peak IPC = 2)

Loop Unrolling

- Replicate loop body to expose more parallelism
  - Reduces loop-control overhead
- Use different registers per replication
  - Called “register renaming”
  - Avoid loop-carried “anti-dependencies”
    - Store followed by a load of the same register
    - Aka “name dependence”
      - Reuse of a register name

Loop Unrolling Example

- Unrolling 4 times
  - Name dependence

```
Loop: ld   x31, 0(x20)      # x31=array[0]
      add  x31, x31, x21    # add scalar in x21
      sd   x31, 0(x20)      # store result
      ld   x31, -8(x20)     # x31=array[1]
      add  x31, x31, x21    # add scalar in x21
      sd   x31, -8(x20)     # store result
      ld   x31, -16(x20)    # x31=array[2]
      add  x31, x31, x21    # add scalar in x21
      sd   x31, -16(x20)    # store result
      ld   x31, -24(x20)    # x31=array[3]
      add  x31, x31, x21    # add scalar in x21
      sd   x31, -24(x20)    # store result
      addi x20, x20,-32     # decrement pointer
      blt  x22, x20, Loop   # branch if x22 < x20
```

Loop Unrolling Example

- Unrolling 4 times
  - Register renaming

```
Loop: ld   x28, 0(x20)      # x28=array[0]
      add  x28, x28, x21    # add scalar in x21
      sd   x28, 0(x20)      # store result
      ld   x31, -8(x20)     # x31=array[1]
      add  x31, x31, x21    # add scalar in x21
      sd   x31, -8(x20)     # store result
      ld   x30, -16(x20)    # x30=array[2]
      add  x30, x30, x21    # add scalar in x21
      sd   x30, -16(x20)    # store result
      ld   x29, -24(x20)    # x29=array[3]
      add  x29, x29, x21    # add scalar in x21
      sd   x29, -24(x20)    # store result
      addi x20, x20,-32     # decrement pointer
      blt  x22, x20, Loop   # branch if x22 < x20
```

Loop Unrolling Example

	ALU/branch	Load/store	cycle
Loop:	addi x20, x20,-32	ld x28, 0(x20)	1
	nop	ld x29, 24(x20)	2
	add x28, x28, x21	ld x30, 16(x20)	3
	add x29, x29, x21	ld x31, 8(x20)	4
	add x30, x30, x21	sd x28, 32(x20)	5
	add x31, x31, x21	sd x29, 24(x20)	6
	nop	sd x30, 16(x20)	7
	blt x22, x20, Loop	sd x31, 8(x20)	8

- IPC = 14/8 = 1.75
  - Closer to 2, but at cost of registers and code size

Compiler Scheduling Limitations

- Scheduling scope
  - Example: can't generally move memory operations past branches
- Limited number of registers (set by ISA)
- Inexact "memory aliasing" information
  - Often prevents reordering of loads above stores by compiler
- Caches misses (or any runtime event) confound scheduling
  - How can the compiler know which loads will miss vs hit?
  - Can impact the compiler's scheduling decisions

Can Hardware Overcome These Limits?

- **Dynamically-scheduled processors**
  - Also called “out-of-order” processors
  - Hardware re-schedules instructions
  - As with pipelining and superscalar, ISA unchanged
    - Same hardware/software interface, apperance of in-order
- Increases scheduling scope
  - Does loop unrolling transparently!
  - Uses branch prediction to “unroll” branches
- Examples:
  - Pentium Pro/II/III (3-wide), Core 2 (4-wide), Alpha 21264 (4-wide), MIPS R10000 (4-wide), Power5 (5-wide)

Example: In-Order Limitations #1

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] → r2	F	D	X	M_1	M_2	W							
add r2 + r3 → r4	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor r4 ^ r5 → r6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [r7] → r4		F	D	**p***	**p***	**p***	X	M_1	M_2	W			

- In-order pipeline, three-cycle load-use penalty
  - 2-wide
- Why not the following?

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] → r2	F	D	X	M_1	M_2	W							
add r2 + r3 → **r4**	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor **r4** ^ r5 → r6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [r7] → **r4**		F	D	**X**	**M_1**	**M_2**	**W**						

Example: In-Order Limitations #2

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] → r2	F	D	X	M_1	M_2	W							
add r2 + r3 → r4	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor r4 ^ r5 → r6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [r7] → r4		F	D	**p***	**p***	**p***	X	M_1	M_2	W			

- In-order pipeline, three-cycle load-use penalty
  - 2-wide
- Why not the following:

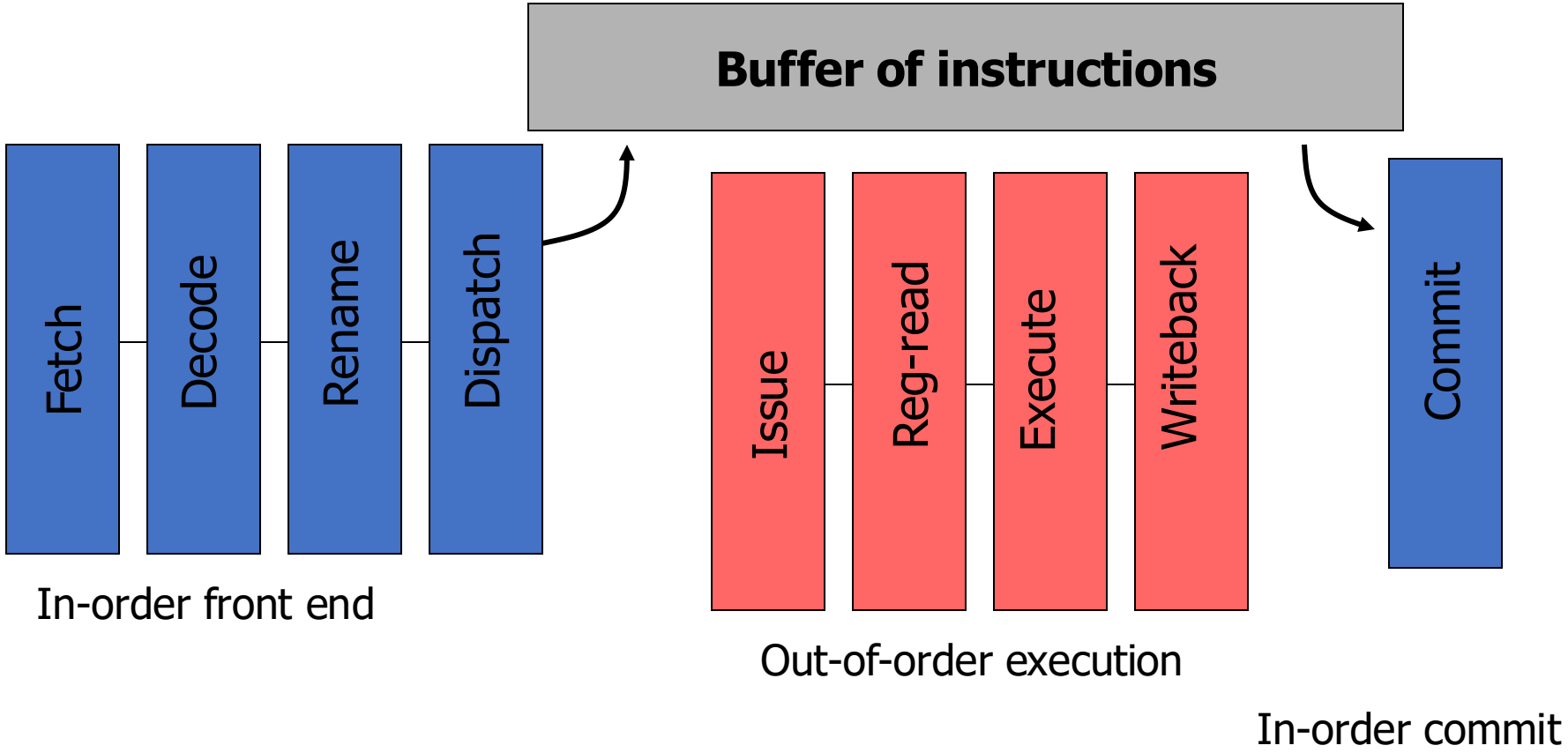
	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] → r2	F	D	X	M_1	M_2	W							
add r2 + r3 → **r4**	F	D	**d***	**d***	**d***	X	M_1	M_2	W				
xor **r4** ^ r5 → r6		F	D	**d***	**d***	**d***	X	M_1	M_2	W			
ld [r7] → **r8**		F	D	**X**	**M_1**	**M_2**	**W**						

Out-of-Order to the Rescue

	0	1	2	3	4	5	6	7	8	9	10	11	12
ld [r1] → r2	F	Di	I	RR	X	M_1	M_2	W	C				
add r2 + r3 → r4	F	Di				I	RR	X	W	C			
xor r4 ^ r5 → r6		F	Di				I	RR	X	W	C		
ld [r7] → r8		F	Di	I	RR	X	M_1	M_2	W		C		

- "Dynamic scheduling" done by the hardware
- Still 2-wide superscalar, but now out-of-order, too
  - Allows instructions to issue when dependences are ready
- Longer pipeline
  - In-order front end: Fetch, "**Dispatch**"
  - Out-of-order execution core:
    - "**Issue**", "**RegisterRead**", Execute, Memory, Writeback
  - In-order retirement: "**Commit**"

Out-of-Order Pipeline



Out-of-Order Execution

- Also called “Dynamic scheduling”
  - Done by the hardware on-the-fly during execution
- Looks at a “window” of instructions waiting to execute
  - Each cycle, picks the next ready instruction(s)
- Two steps to enable out-of-order execution:

  Step #1: Register renaming - to avoid “false” dependencies

  Step #2: Dynamically schedule - to enforce “true” dependencies
- Key to understanding out-of-order execution:
  - **Data dependencies**

Dependence types

- RAW (Read After Write) = “true dependence” (true)
  - mul r0 * r1 → r2
  - …
  - add r2 + r3 → r4
- WAW (Write After Write) = “output dependence” (false)
  - mul r0 * r1 → r2
  - …
  - add r1 + r3 → r2
- WAR (Write After Read) = “anti-dependence” (false)
  - mul r0 * r1 → r2
  - …
  - add r3 + r4 → r1
  - WAW & WAR are “false”, Can be totally eliminated by “renaming”

Static & Dynamic Scheduling

- Code scheduling
  - To reduce pipeline stalls
  - To increase ILP (insn level parallelism)
- Static scheduling by the compiler
  - Approach & limitations
- Dynamic scheduling in hardware
  - Register renaming
  - Instruction selection
  - Handling memory operations